PROGRAM 4 4.c

Sparse Matrix Addition

C SOURCE CODE 4.c

```c
/* Sparse Matrix Addition */

#include <stdio.h>

int main() {
    int rows, cols, non_zero1, non_zero2;
    printf("Enter total number of rows and columns for the matrices: ");
    scanf("%d%d", &rows, &cols);
    printf("Enter number of non-zero elements for Matrix 1: ");
    scanf("%d", &non_zero1);
    printf("Enter number of non-zero elements for Matrix 2: ");
    scanf("%d", &non_zero2);

    int sparse1[non_zero1 + 1][3];
    int sparse2[non_zero2 + 1][3];
    int sum[non_zero1 + non_zero2 + 1][3];

    sparse1[0][0] = rows; sparse1[0][1] = cols; sparse1[0][2] = non_zero1;
    sparse2[0][0] = rows; sparse2[0][1] = cols; sparse2[0][2] = non_zero2;
    sum[0][0] = rows; sum[0][1] = cols;

    printf("\nEnter Triplets for Matrix 1 (Row, Column, Value):\n");
    for (int i = 1; i ≤ non_zero1; i++) {
        scanf("%d%d%d", &sparse1[i][0], &sparse1[i][1], &sparse1[i][2]);
    }

    printf("\nEnter Triplets for Matrix 2 (Row, Column, Value):\n");
    for (int i = 1; i ≤ non_zero2; i++) {
        scanf("%d%d%d", &sparse2[i][0], &sparse2[i][1], &sparse2[i][2]);
    }

    int i = 1, j = 1, k = 1;

    while (i ≤ non_zero1 && j ≤ non_zero2) {
        if (sparse1[i][0] == sparse2[j][0]) {
            if (sparse1[i][1] == sparse2[j][1]) {
                sum[k][0] = sparse1[i][0];
                sum[k][1] = sparse1[i][1];
                sum[k][2] = sparse1[i][2] + sparse2[j][2];
                i++; j++; k++;
            }
            else if (sparse1[i][1] < sparse2[j][1]) {
                sum[k][0] = sparse1[i][0];
                sum[k][1] = sparse1[i][1];
                sum[k][2] = sparse1[i][2];
                i++; k++;
            }
            else {
                sum[k][0] = sparse2[j][0];
                sum[k][1] = sparse2[j][1];
                sum[k][2] = sparse2[j][2];
                j++; k++;
            }
        }
        else if (sparse1[i][0] < sparse2[j][0]) {
            sum[k][0] = sparse1[i][0];
            sum[k][1] = sparse1[i][1];
            sum[k][2] = sparse1[i][2];
            i++; k++;
        }
        else {
            sum[k][0] = sparse2[j][0];
            sum[k][1] = sparse2[j][1];
            sum[k][2] = sparse2[j][2];
            j++; k++;
```

```
        }
    }

    while (i ≤ non_zero1) {
        sum[k][0] = sparse1[i][0];
        sum[k][1] = sparse1[i][1];
        sum[k][2] = sparse1[i][2];
        i++; k++;
    }

    while (j ≤ non_zero2) {
        sum[k][0] = sparse2[j][0];
        sum[k][1] = sparse2[j][1];
        sum[k][2] = sparse2[j][2];
        j++; k++;
    }

    sum[0][2] = k - 1;

    printf("\nResultant Sparse Matrix (Triplet Representation):\n");
    printf("Row\tCol\tValue\n");
    for (int m = 0; m < k; m++) {
        printf("%d\t%d\t%d\n", sum[m][0], sum[m][1], sum[m][2]);
    }

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
Enter total number of rows and columns for the matrices: 3 3
Enter number of non-zero elements for Matrix 1: 2
Enter number of non-zero elements for Matrix 2: 2

Enter Triplets for Matrix 1 (Row, Column, Value):
0 0 5
1 1 3

Enter Triplets for Matrix 2 (Row, Column, Value):
0 0 2
2 2 4

Resultant Sparse Matrix (Triplet Representation):
Row     Col     Value
3       3       3
0       0       7
1       1       3
2       2       4
```

PROGRAM 11 11.c

Evaluation of Postfix Expression Using Stack

C SOURCE CODE

11.c

```c
#include<stdio.h>
#include<ctype.h>
#include<math.h>
#define MAX 100
int stack[MAX];
int top=-1;
void push(int x)
{
stack[++top]=x;
}
int pop()
{
return stack[top--];
}
int main()
{
char postfix[MAX];
int i,a,b,result;
printf("Enter the postfix expression: \n");
scanf("%s",postfix);
for(i=0;postfix[i]≠'\0';i++)
{
if (isdigit(postfix[i]))
{
push(postfix[i]-'0');
}
else{
b=pop();
a=pop();
switch(postfix[i])
{
case'+':
result=a+b;
break;

case'-':
result=a-b;
break;

case'*':
result=a*b;
break;

case'/':
result=a/b;
break;

case'%':
result=a%b;
break;

case'^':
result=a^b;
break;
}
push(result);
}}
printf("Result: %d\n",pop());
return 0;
}

```

PROGRAM OUTPUT

Execution Verification

```
Enter the postfix expression: 2 3 + 4 5 - *
Result: -5
```